Information Sciences 676 (2024) 120855

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

An adaptive evolutionary search-based method for efficiently tackling the set-union knapsack problem

Juntao Zhao [a,b], Mhand Hifi [b,*]

[a] College of Information Science and Engineering, Northeastern University, Shenyang, 110819, PR China
[b] EPROAD UR 4669, UPJV, 7 rue du Moulin Neuf, Amiens, 80000, France

ARTICLE INFO

Keywords:
Evolutionary
Heuristic
Knapsack
Optimization
Scatter

ABSTRACT

This paper tackles the NP-hard set-union knapsack problem using a novel adaptive evolutionary algorithm. Such a problem has applications in database query optimization, network design, and project portfolio management. The method starts by creating an archive set with elite and diversified solutions, employing a customized procedure based on item scores and Hamming max-min distance for diversity. Various operators are added to enhance solution quality in the elite set and refine the search process. To counteract premature convergence, combination and subset reference update methods are used. The fusion strategy serves as an exploration mechanism, generating new solutions and preventing premature convergence. Finally, the effectiveness and performance of the proposed method is evaluated on a set of benchmark instances taken from existing literature and by conducting a thorough statistical analysis of all achieved results. All obtained results are systematically compared against those achieved by state-of-the-art methods, indicating a substantial improvement of 50% for the large-sized instances, outperforming those published in the literature.

1. Introduction

The knapsack problem is extensively discussed in [15], where it plays a central role in various practical contexts. It frequently arises in resource allocation scenarios, with its primary objective being to reconcile multiple technological constraints while optimizing financial objectives. Such a classical problem has not only motivated the development and exploration of numerous related variants across diverse domains but has also found applications in various fields.

For instance, in the domain of cryptography [19], a knapsack problem variant was explored for encryption and security applications. In logistics [22], strategies inspired by the knapsack problem are employed to optimize cargo and supply chain management. Similarly, in multimedia [14], knapsack-inspired approaches are widely utilized for optimizing data storage and retrieval, achieving a balance between data prioritization and transmission efficiency. For the binary knapsack problem, a recently tailored differential evolution algorithm [2] was meticulously designed. In experiments, the authors highlighted its effectiveness in discovering solutions that outperform the current best ones for large knapsack problems. This observation accentuates the continued relevance of evolutionary approaches, particularly when intelligently integrated with robust neighborhood search methods.

Furthermore, the knapsack problem has extended its influence into other variants, such as the set union knapsack problem, a more complex variant of the classical knapsack problem. In this variant, the objective is to select subsets of items that maximize

* Corresponding author.
E-mail addresses: juntao.zhao@u-picardie.fr (J. Zhao), hifi@u-picardie.fr (M. Hifi).

https://doi.org/10.1016/j.ins.2024.120855
Received 20 February 2024; Received in revised form 13 April 2024; Accepted 29 May 2024

Available online 3 June 2024
0020-0255/© 2024 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

utility while accommodating constraints on the total capacity of elements from each set. This variation has been instrumental in diverse applications, including database query optimization, network design, and project portfolio management. A practical situation illustrating the set union knapsack problem relates to resource allocation problems, such as optimizing routing in computer networks [16]. In this context, consider a network topology containing multiple nodes, representing routers or servers, interconnected by links characterizing communication channels. Each node offers unique services with varying values and resource demands (e.g., bandwidth, processing capacity). The goal of the problem is to select a subset of nodes to maximize the total value of services provided, while ensuring that the overall resource consumption does not exceed predefined limits (e.g., bandwidth, processing capacity). Such a situation corresponds to the set union knapsack problem, wherein each node represents an item with an associated profit, and the services it offers represent the elements within the item, each with its specific resource requirements. The capacity constraint of the knapsack reflects the overall resource consumption limit of the network.

In recent developments, the approach to addressing the set-union knapsack problem has encountered a refined evolution. Such an evolution is characterized by the strategic integration of diverse metaheuristics with standard operators, with a particular emphasis on this integration during the exploitation phase. The introduction of randomness into the exploration of search spaces imparts a stochastic dimension to these approaches, explicitly aiming to enhance the potential for diversification in addressing the set-union knapsack problem. This problem has captured significant attention from researchers due to its complexity and practical relevance, particularly in fields such as data management, where the precise selection of sets is crucial for efficient resource utilization. Furthermore, the inherent complexity of the set-union problem has stimulated the development of advanced algorithms, establishing it as a central focus within the expansive domain of combinatorial optimization.

1.1. The problem

An instance of P_{SUK} is defined by two sets: a set U that includes elements, i.e., $U = \{1, \ldots, n\}$, and a set I that contains items, i.e., $I = \{1, \ldots, m\}$. Further, each item $i \in I$ corresponds to a subset of elements, denoted as I_i, and is associated with a nonnegative profit $p : I \to R^+$, while each element is characterized by a nonnegative weight $w : U \to R^+$. The objective of the problem is to identify a subset $I^\star$ of items, where $I^\star \subseteq I$, such that the overall profit $z(I^\star)$ is maximized without exceeding the knapsack capacity b. In fact, the weights associated with an element are counted once, and an element is selected for inclusion in the solution if it belongs to at least one selected item. Conversely, elements belonging to items not included in the current feasible solution are excluded. Each exclusion constraint can be formulated as follows: Let $\mathrm{d}_j = \sum_{i=1}^{m} a_{ij} x_i$ where $a_{ij} = 1$ if j belongs to I_i (indicating that at least one selected item in the solution contains j), and 0 otherwise. Then, y_j is set to 1 if $\mathrm{d}_j \geq 1$, and 0 otherwise [3,4].

Consequently, a formal description of the problem, P_{SUK}, can be stated as follows:

$$\max \quad z = \sum_{i=1}^{m} p_i x_i \tag{1}$$

$$\text{s.t.} \quad \sum_{j=1}^{n} w_j y_j \leq b \tag{2}$$

$$x_i \leq y_j \qquad i \in I,\ j \in I_i, \tag{3}$$

$$y_j \leq \min \left\{ 1, \sum_{i=1}^{m} a_{ij} x_i \right\} \qquad i \in I,\ j \in I_i, \tag{4}$$

$$x_i \in \{0,1\},\ y_j \in \{0,1\} \qquad i \in I,\ j \in I_i, \tag{5}$$

where x_i (resp. y_j) represents the binary decision variable for item $i \in I$ (resp. element related to $i \in I, j \in I_i$), conveying whether they are part of the solution or not. Inequality (2) denotes the knapsack constraint with a capacity b that is related to the elements in set U. Inequalities (3) and (4) indicate that for each item i, there are several precedence constraints. These constraints ensure that whenever an item is included in a solution, its elements are automatically assigned a value of one. Conversely, if certain elements belong to a solution, it's possible for some of their corresponding items not to be assigned a value of one. Finally, Inequality (5) is related to each item i and each element j, ensuring that the decision variables are binary (0 or 1). When $x_i = 1$, it signifies that the item i is included in the knapsack; otherwise, it is 0. Correspondingly, $y_j = 1$ if at least one item belonging to the current solution contains j, and $y_j = 0$ otherwise. The objective function $z(x, y)$ (Eq. (1)) is linear and calculates the sum of the profits of the selected (or not selected) items in the solution.

1.2. Contribution

In this paper, an adaptive scatter search is tailored for solving the set-union knapsack problem; that is an adaptive evolutionary search-based method. The proposed algorithm can generate the archive set based on the adaptive scoring procedure. It means that when a new item is added to the solution set S, the scores of non-selected items will be recalculated according to the elements already belonging to the current solution S. Additionally, the subset is updated using the Hamming max-min distance, and the archive set can be adaptively updated based on the combination of the dynamic reference set and the diversified set. Specifically, the designed method incorporates the following features:

1. Initiate the search process by generating an adaptive *tailored greedy solution* through the application of the *scoring procedure* (cf., Section 3.2.1). During this phase, an archive set is created, containing a subset of elite solutions along with a complementary subset of diversified solutions. The exploration covers a segment of the solution space, using the Hamming max-min distance.
2. Conduct a *series of shakings* to *exploit* the solutions within the reference set (cf., Section 3.2.2). During this phase, a multi-tabu procedure is considered for exploiting and highlighting the elite solutions, using both adding and moving operators.
3. Regenerate the archive set by *hybridizing both the combination method and the reference update method* (cf., Section 3.2.4). During the hybridization, a series of solutions are combined using the fusion operator, and the resulting solutions are further improved using the tailored greedy solution.

The last two steps, which involve the combination of various operators, are integrated into an iterative process. Within this process, exploitation and exploration strategies collaborate to refine the solutions within the elite subset and diversify the solutions of the complementary subset until a stable final set of solutions is reached (or according to the fixed runtime limit), from which the best solution is extracted as a (sub)optimal solution for P_{SUK}.

The structure of the paper is planned as follows. Section 2 reviews prior research on the set-union knapsack problem (P_{SUK}). Section 3 details the proposed method, which is an adaptive evolutionary search-based approach, incorporating a tailored scatter search with various new operators that contribute to effectively exploiting and exploring solutions within a given archive set. Section 4 evaluates the behavior of the proposed algorithm on benchmark instances extracted from the literature, comparing its results with those achieved by state-of-the-art methods available in the literature. Finally, Section 5 summarizes the main contributions of this work and provides perspectives for extending the search of the studied problem and its variants.

2. Background

Due to the inherent complexity of P_{SUK}, the predominant approach for addressing this problem involves proposing approximate solution methods, often incorporating heuristics and metaheuristics.

To the best of our knowledge, there are limited studies in the literature that have explored exact methods. For instance, a dynamic programming approach has been considered in [12], transforming the problem into the densest k-subgraph problem. The authors' method proves most effective for small-sized instances, especially those with moderate knapsack capacity constraints.

In contrast, recent years have seen the emergence of various studies employing diverse metaheuristic techniques for tackling P_{SUK}. In [3], the authors introduced an approximation algorithm based on a greedy procedure. The algorithm achieves an approximation ratio of $\frac{1}{(1-e^{(-1/d)})}$, where d (with $d \geq 2$) represents an upper limit associated with the number of occurrences of all elements in the problem.

In [13], a binary artificial bee colony algorithm equipped with a specific mapping function to tackle P_{SUK} has been introduced. The approach involved a hybrid strategy, combining an evolutionary operator-based greedy procedure to enhance the current solutions. The method was evaluated on some benchmark instances extracted from the literature to determine its competitiveness.

A binary swarm intelligence method has been devised in [20], integrating genetic algorithms and particle swarm optimization, eliminating the necessity for specific local search steps. The approach significantly enhanced the previously established upper bounds, especially when compared to bounds achieved by alternative methods in the existing literature.

In [17], the authors presented a hybrid approach combining binary particle swarm optimization (BPSO) with tabu search (TS) to tackle P_{SUK}. The proposed HBPSO/TS algorithm employs adaptive penalty functions, refined position updating rules, tabu-based mutation procedures, and tabu search techniques to improve both solution quality and computational efficiency. Experimental evaluations conducted on benchmark instances reveal HBPSO/TS's superiority over existing algorithms, demonstrating its effectiveness in solving P_{SUK} instances.

In [26], the double Jaya algorithm, an enhanced hybrid approach, was proposed. It combines intensification and diversification by integrating the standard Jaya procedure with differential evolution, along with specialized operators. Evaluation on benchmark instances demonstrated its ability to surpass or match existing methods in terms of performance.

In [23], the authors developed an iterated two-phase local search algorithm, including an exploration phase for intensification and an escaping phase for diversification. It was evaluated on thirty benchmark instances, demonstrating its ability to improve upon existing best solution values in the literature.

In [24], a kernel-based tabu search algorithm tailored for P_{SUK} was designed. It demonstrated superiority over current algorithms in solution quality, robustness, and computation time, with their approach outperforming others, particularly highlighting its effectiveness in managing extensive instances containing a minimum of 500 items and elements. The algorithm, requiring only three parameters in contrast to existing methods, emerges as a practical and efficient tool for solving P_{SUK}.

A reinforcement learning strategy that combines particle swarm optimization, genetic algorithm, and genetic-based swarm as optimizers for binary optimization problems has been designed in [21]. With a focus on promoting efficient optimizers and enhancing diversity to avoid local optima, demonstrating significant performance improvements on the set-union knapsack problem.

Finally, other works tackled P_{SUK} and other related problems with various methods: particle swarm optimization [6], the binary version of the grey wolf optimization algorithm [11], the artificial search agents with cognitive intelligence [20], the iterative rounding strategy-based algorithm [4], the population-based method combined with path-relinking [5], and others [1].

To better organize existing research on the P_{SUK} problem, Table 1 provides a summary of previous studies, including their primary contributions and characteristics. Additionally, the table serves to clarify the diverse approaches and methodologies employed in tackling the P_{SUK} problem across various research areas. From Table 1, it is evident that existing research predominantly relies

Table 1
Comprehensive overview of related works and the main contributions for solving P_{SUK}.

References	Methods	Search strategies	Instance size	Categories
Goldschmidt et al. [12]	Dynamic Programming	Transforming the original problem into densest subgraph problem	Only for special and small-sized instances	Exact
Arulselvan [3]	Greedy algorithm	Based on the approximation ratio	–	Heuristic
He et al. [13]	BABC	Full mapping function Greedy repair strategy Evolutionary framework	Within 85 to 500	Metaheuristic
Ozsoydan & Baykasoglu [20]	gPSO	Local search procedure Optional mutation operator	Within 85 to 500	Metaheuristic
Lin et al. [17]	HBPSO/TS	Penalty function technique Tabu based mutation procedure Tabu search procedure	Within 85 to 500	Heuristic
Wu & He [26]	DHJaya	Mapping function and Cauchy mutation Combined with DE Repairing and optimization	Within 85 to 500	Metaheuristic
Wei & Hao [23]	I2LS	Two phases local search VND and tabu search Frequency-based perturbation	Within 85 to 500	Heuristic
Wei & Hao [24]	KBTS	Dynamic initialization Kernel and non-kernel search Tabu strategy	Within 85 to 1000	Heuristic
Gölcük & Ozsoydan [11]	bGWO	Directly search the binary space Adaptive inheritance mechanisms	Within 85 to 500	Metaheuristic
Dahmani et al. [6]	HSPO-BA	Reactive strategy 2-opt operator	Within 85 to 1000	Heuristic
Dahmani et al. [4]	IRSP	Linear relaxation Iterative rounding strategy Degrading and re-optimization	Within 85 to 1000	Heuristic
Dahmani et al. [5]	PSO-B	Iterative swarm optimization Local search based on critical item Path-relinking	Within 85 to 1000	Heuristic
Ozsoydan & Gölcük [21]	PSO, GA and gbPSO	Q-learning Random immigrants mechanism Mutation procedure	Within 85 to 500	Heuristic and Machine learning

on swarm intelligence frameworks or specific solution-based frameworks. In comparison, the proposed method distinguishes itself through the adoption of a hybrid scatter search framework employing an adaptive tailored constructive procedure combined with a cooperative multi-neighborhood tabu search strategy, effectively addressing the P_{SUK} problem. On the other hand, it is noted that most of these algorithms are primarily validated on small and medium-sized instances, which suggests a lack of robustness as the instance size increases. As discussed in Section 4, the proposed method proves highly effective in solving P_{SUK} instances and competes favorably with state-of-the-art algorithms presented in [26,4,23,24,6,5] across a set of 84 benchmark instances commonly tested in the literature.

3. A tailored adaptive evolutionary algorithm

This section consists of three parts. The first part (Section 3.1) discusses the basic version of the evolutionary algorithm, while the second part (Section 3.2) introduces the tailored adaptive evolutionary method. In this method, various strategies are incorporated to enhance the search process, aiming to highlight the final (sub)optimal solution. Finally, the third part (Section 3.3) provides a computational complexity analysis of the proposed algorithm.

3.1. The basic evolutionary search

The Scatter Search (SS) is a population-based evolutionary algorithm [7,18] that relies on a "reference set" with diverse elite solutions and a complementary subset with diversified solutions. Such an approach can be considered as an extended version of the genetic algorithm, incorporating multiple operators to guide the generation of both high-quality and diversified solutions. The method operates iteratively, and its key characteristics are outlined in Algorithm 1, detailing its general steps.

Algorithm 1 Scatter Search (SS).

Input. A instance of P_{SUK}.
Output. An archive set $\mathcal{P}$ containing the reference set S_{Ref}, and the diversified set S_{Div}.

1: Set $\mathcal{P} = \emptyset$.
2: Apply the *"generation method"* for starting the archive set $\mathcal{P}$.
3: **for** $S_i \in \mathcal{P}$ **do**
4: Call the *"improvement method"* for reaching an enhanced solution S_{new}.
5: **if** $|\mathcal{P}| <$ PopSize and $S_{new} \notin \mathcal{P}$ **then**
6: $\mathcal{P} = \mathcal{P} \cup \{S_{new}\}$
7: **end if**
8: **end for**
9: Select a subset of best solutions in $\mathcal{P}$ to build the initial reference set S_{Ref} according to the order of objective function values.
10: **repeat**
11: Call the *"subset generation method"* for reaching a series of diversified solutions *NewSubsets*.
12: **while** *NewSubsets* $\neq \emptyset$ **do**
13: Call the *"solution combination method"* for combining the overall solutions belonging to the reference set.
14: Apply the *"improvement method"* to the new solutions.
15: Apply the *"reference update method"* for highlighting the current reference set.
16: **end while**
17: **until** (a predefined condition is performed)
18: **return** $\mathcal{P}$

A key component of the evolutionary algorithm (Algorithm 1) is the archive set $\mathcal{P}$, which consists of two subsets according to the problem being solved: the reference set S_{Ref} containing the solutions with the highest objective values, and the complementary set S_{Div} containing diversified solutions. The solutions in S are carefully selected to cover a wide range of the search space, and in some cases, it contains feasible and/or unfeasible solutions. They serve as a basis for generating new candidate solutions through various search and combination strategies.

By maintaining the high-quality reference set and the complementary set of diversified solutions, the evolutionary method can efficiently explore the search space and find promising solutions. The method starts (line 2) by applying the "generation method" to create an initial set of randomly diversified solutions, which is then divided into two complementary sets. As recommended in the literature [7,18], the size of the archive set $\mathcal{P}$ is typically at least five times the size of the reference set S_{Ref}. Hence, for each solution S in $\mathcal{P}$, the "improvement method" is employed to enhance the quality of candidate solutions (line 4). Moreover, if the new solution S_{new} does not belong to $\mathcal{P}$ and the size of $\mathcal{P}$ is smaller than the predetermined "PopSize", the new solution S_{new} is added to $\mathcal{P}$ (lines 5 to 7). Subsequently, the initial reference set is constructed based on the order of objective function values (line 9). This step includes selecting a set of distinct and maximally diverse solutions from $\mathcal{P}$. The main loop of the method, denoted by the `repeat` keyword (lines 10 to 17), generates a series of new enhanced reference sets and diversified complementary sets. The loop consists of four stages that are applied iteratively:

1. **Subset generation method.** The algorithm uses the "subset generation method" to create new subsets of solutions (*NewSubsets*) by applying various operators. In this case, the method combines the reference solutions or the best reference solutions with the remaining solutions in the reference set.
2. **Solution combination method.** It involves creating a linear combination between each subset of solutions generated by the "subset generation method". This process creates either the best solution or a subset of solutions resulting from the combination.
3. **Improvement method.** The algorithm extends the solutions of the reference set by applying the "improvement method". It serves the purpose of either enhancing the quality of the current feasible solution or correcting the solution to make it feasible.
4. **Reference update method.** It is used to store a limited sub-population that contains the best solutions from the population. At this step of the algorithm, the updated set is generated by using a threshold strategy that determines which solutions are accepted to be included in the current reference set.

The algorithm continues to iterate through the aforementioned stages until a stopping criterion is matched, such as a maximum number of iterations or a satisfactory solution is reached within a specified runtime limit.

3.2. The proposed adaptive evolutionary method

3.2.1. A tailored generation method

The starting reference set denotes the initial group of solutions used as a foundation for applying selection and sorting strategies in optimization algorithms. Such a starting reference set may be derived by employing a greedy procedure that rapidly provides solutions, prioritizing local improvements without an evaluation of wider global implications.

The initialization procedure in the proposed method involves a greedy approach to obtain its reference set, consisting of a series of generated feasible solutions. It is based on the principle that, for a given solution S, the weight of each element is counted only once. When adding a new item x_t to S, only the new elements of the item that do not already belong to S will affect the total weight. Therefore, a specific score (profit-per-weight ratio) can be defined for each item as follows:

$$Score(x_t) = \frac{p_t}{\sum_{j \in I_t \wedge j \notin \cup_{i \in S} I_i} w_j}, \ t = 1, \ldots, |I| \tag{6}$$

Algorithm 2 Tailored Procedure (TP).

Input. An instance of P_{SUK}.
Output. A feasible solution S for P_{SUK}.

1: Set $\mathrm{P}^{r}_{SUK} = \mathrm{P}_{SUK}$
2: For the current problem P^{r}_{SUK}, rank all items in non-ascending order of their score, i.e., $s_1 \geq s_2 \geq \cdots \geq s_m$, and set $\bar{b} = b$, $\bar{I} = I$, and $S = \emptyset$.
3: For all $i \in \bar{I}$, if $\omega_i > \bar{b}$, remove these items from $\bar{I}$.
4: If $\bar{I} = \emptyset$, then stop with the best solution S.
Let $c = \arg\max_{i \in \bar{I}} \{s_i\}$.
1. Set $S = S \cup \{x_c\}$, $\bar{I} = \bar{I} \setminus \{x_c\}$, and $\bar{b} = \bar{b} - \omega_c$.
2. Recalculate overall potential weights of all items belonging to $\bar{I}$.
3. If $\bar{I} \neq \emptyset$, then order the items of $\bar{I}$ according to the non-ascending order of the score.
5: Iterate both steps (3) and (4).
6: **return** S

Specifically, the method initiates by calculating the corresponding score for each item i. Subsequently, all items are ranked in non-ascending order based on their score, i.e.,

$$s_1 \geq s_2 \geq \cdots \geq s_m \tag{7}$$

The procedure adapts to the induced instance by recalculating the score of non-selected items based on the elements belonging to the current solution S after adding a new item. It computes the dynamic score of each non-selected item and adds the item with the highest score to S iteratively until either all items are examined or the residual value related to the knapsack capacity is insufficient for including a remaining item with the smallest weight.

Algorithm 2 describes the main steps of the Tailored Procedure, denoted as TP. Adding a new item i (setting $x_i = 1$), where $i = 1, \ldots, m$, to the current solution S leads to a reduced (sub)problem, denoted as P^{r}_{SUK}. In this scenario, removing elements y_i ($j = 1, \ldots, n$) from P_{SUK} eliminates the need to consider all possible combinations of these elements. To address this, updating w_i based on the currently selected item is proposed. Consequently, solving P_{SUK} can be accomplished by applying the principles of the greedy procedure.

Now, Algorithm 2 generates a single solution for the problem. To establish a diverse reference set of solutions, a stochastic approach is adopted using the Augmented TP (ATP) procedure. A reference set of size M is formed through a loop where, at each step, a random item i (associated with the decision variable x_i) is selected from the overall sorted items of the set I. TP is applied to achieve a feasible solution, ensuring that each item has an opportunity to be part of at least one feasible solution. Finally, rank all the solutions of the reference set in descending order of their objective values (for a maximization problem).

Upon obtaining a new solution, it is either included in the reference set if it does not already belong to the current set of solutions or replaces one solution with the smallest objective value. If the reference set remains incomplete, the random selection can be expanded to include pairs of items, triplets, and so on. The initialization procedure concludes either with a reference set of M different solutions or one with fewer solutions. In the latter case, the algorithm completes that reference set when the search process begins.

3.2.2. Employing an improvement method

The so-called "improvement method", often considered a crucial step in SS, involves highlighting the quality of the diversified solutions, belonging to the reference set, by applying local search or other enhancement operators. It can also be employed to enhance any given solution. In various scenarios, it has been observed that local searches often use the $k-$opt operator to explore the existing subspace of solutions, serving as a means to avoid overloading resolution methods.

A) Applying a deep exploration In this work, a deep exploration strategy is introduced to improve the positioning of the feed based on the current best solution, called the *target solution* and denoted as S_{Best}. The objective is to guide all solutions toward the target solution using a jumping principle through diverse subspaces of solutions. While simple local searches could conduct such exploration, it is often observed that procedures are prone to getting trapped in local optima. To overcome the aforementioned limitation and more effectively explore the search subspaces, a Multi-neighborhood-based Tabu Search strategy (denoted MTS) is employed to enhance the quality of the current solution. The goal of the proposed strategy involves two aspects: (i) minimizing the risk of being confined to local optima, resulting in significant runtime reduction, and (ii) expanding the exploration of the solution space.

The approach (described in Algorithm 3) simultaneously uses two different neighborhoods:

1. The first neighborhood, noted N_+, is employed for adding an object, and
2. The second operator, noted $N_{\leftrightarrow}$, is used for making a permutation between two different objects.

These neighborhoods work as follows. Let $S^{(1)}$ and $\bar{S}^{(1)}$ be respectively the sets of selected items and unselected ones such that any solution S can be represented by $S =< S^{(1)}, \bar{S}^{(1)} >$. The neighborhood N of the initial solution S can be defined using the adding operator (noted $N_+(S)$) and swapping/permutation operator ($N_{\leftrightarrow}(S)$), which can be described as follows:

Algorithm 3 Multi-neighborhood Tabu Search (MTS) for P_{SUK}.

Input. A feasible solution S, the maximum number of iterations $IterMax$.
Output. An improved local optimum solution S'_{SUK}.

$iter \leftarrow 0$
$S'_{SUK} \leftarrow S$
while $\left(iter \leq Iter_{\text{Max}}\right)$ **do**
  // $N(S)$ is related to Eqs. (8) and (9), and the neighborhoods evaluated according to equations from (10) to (12)
  $S' \leftarrow \arg\max\left\{z(S), S \in N_{+}(S) \cup N_{\leftrightarrow}(S)\right\}$
  $S \leftarrow S'$
  if $\left(z(S) > z(S'_{SUK})\right)$ **then**
    $S'_{SUK} \leftarrow S$ and $iter \leftarrow 0$
  else
    $iter \leftarrow iter + 1$
  end if
  Update tabu list and tabu tenure
end while
return S'_{SUK}.

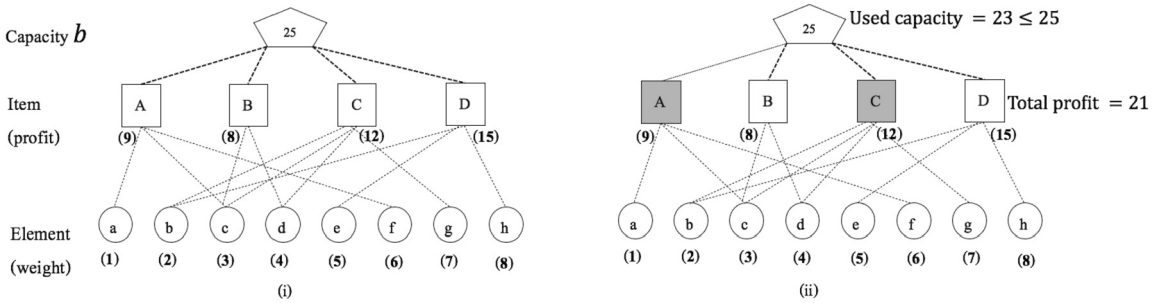


Fig. 1. Illustration of (i) an instance of the set-union knapsack problem, where $U = \{a, b, c, d, e, f, g, h\}$ and $I = \{A, B, C, D\}$, and (ii) a given feasible solution.

$$N_{+}(S) = \left\{ S \cup \{u\}, u \in \bar{S}^{(1)} \text{ such that } \sum_{j=1}^{n} w_j \bar{y}_j + w_u \leq b \right\} \tag{8}$$

and

$$N_{\leftrightarrow}(S) = \left\{ S \cup \{u\} \setminus \{v\}, v \in S^{(1)}, u \in \bar{S}^{(1)} \text{ such that } \sum_{j=1}^{n} w_j \bar{y}_j + w_u - w_v \leq b \right\} \tag{9}$$

such that the constraints of the original problem related to Eq. (2) is satisfied.

Fig. 1 illustrates an instance of P_{SUK} (on the left) with a given starting feasible solution S (on the right). Indeed, on one hand, Fig. 1.(i) displays the instance, where data are represented as follows: $U = \{a, b, c, d, e, f, g, h\}$, $I = \{A, B, C, D\}$, item profits $p = \{9, 8, 12, 15\}$, element weights $w = \{1, 2, 3, 4, 5, 6, 7, 8\}$, knapsack capacity $b = 25$, and item subsets $I_A = \{a, c, f\}$, $I_B = \{c, d\}$, $I_C = \{b, c, d, g\}$, and $I_D = \{b, e, h\}$, respectively. On the other hand, Fig. 1.(ii) shows the solution $S = \{1, 0, 1, 0\}$, which means that the first (A) and the third (C) items are selected, and the complementary set of items not fixed in the current solution is noted $\bar{S}^{(1)} = \{B, D\}$. The current solution S provides an objective value of $z(S) = p_A + p_C = 21$, while the used aggregate weight related to elements a, b, c, d, f, and g in S is equal to $W(S) = w_a + w_b + w_c + w_d + w_f + w_g = 23$ (of course, we recall that each element's weight is counted once whenever that element appears several times in the solution at hand).

Fig. 2 illustrates the functionality of both employed operators:

Adding operator. On one hand, Fig. 2.(i) displays the resulting feasible solution provided by using the adding operator: an item $B \in \bar{S}^{(1)}$ is added to the feasible solution S, resulting in a new solution $S' = \{1, 1, 1, 0\}$; that is a solution composed of items A, B, and D, respectively. The weight related to S' remains unchanged (according to that of S) since the elements c and d belonging to the item B are already included in the solution S. According to the principle of counting each element only once, the aggregate weight is equal to $W(S') = W(S) = 23$. However, the objective value of the achieved solution becomes $z(S') = p_A + p_B + p_C = 29$, which means that $z(S') > z(S)$ (S' is better than S).

Swapping operator. On the other hand, Fig. 2.(ii) reports the solution $S' = \{1, 0, 0, 1\}$ provided when applying a swapping of two items; they are between $C \in S^{(1)}$ and $D \in \bar{S}^{(1)}$ according to the starting solution S. In this case, the objective value of the resulting solution S' is equal to $z(S') = p_A + p_D = 24$, while its aggregate weight is equal to $W(S') = w_a + w_b + w_c + w_e + w_f + w_h = 25$ (computed according to the weights related to elements a, b, c, e, f, and h, respectively). Thus, one can also observe that $z(S') = 24 > z(S) = 21$, indicating that the swapping operator is capable of providing a better solution.

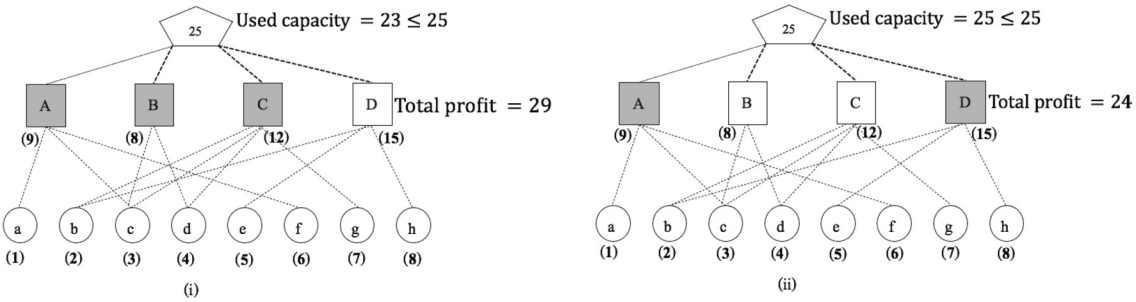


Fig. 2. Illustration the use of both incorporated operators: (i) the adding operator, and (ii) the swapping operator.

B) Highlighting the search process with tabu search Furthermore, an efficient neighborhood evaluation method, the *gain updating strategy*, is employed. This method relies on the maintenance of the Δ matrix [8,9,25]. Such a method enables straightforward addition and subtraction operations instead of repetitive multiplications to compute the profit value related to the vector (x, y). In this case, S' represents the added/permuted neighbor solution of the current solution S, simplifying the process as follows:

$$P_i = \sum_{i=1}^{m} p_i \bar{x}_i, i \in I \tag{10}$$

$$z(S'_{+}) = P_i + p_u \quad \text{and} \quad z(S'_{\leftrightarrow}) = P_i + p_u - p_v \tag{11}$$

such that

$$\Delta = z(S') - z(S). \tag{12}$$

Remark. In tabu search, a tabu list prevents revisiting solutions for a specified number of iterations. The algorithm selects the best neighbor solution from those permitted by the tabu list to replace the current solution. Even if a neighbor solution is restricted by the tabu list, it is still considered if it offers an improvement over the best solution found thus far. This principle, known as the aspiration criterion [10], highlights the algorithm's adaptability.

In this work, whenever either the N_+ or $N_{\leftrightarrow}$ operator is applied, each item i involved in the add/permute operator is added to the tabu list, restricting them from moving away from their respective item set for the subsequent T consecutive iterations. In this case, T is denoted as the tabu tenure [24]. After a series of iterations, the (optimal) local solution for the current configuration is determined by selecting the best neighborhood based on the values of Δ (we recall that the main steps of the Modified Tabu Search (MTS) are summarized in Algorithm 3).

3.2.3. Updating the subsets

Frequently, SS employs a sequence of combinations, as described in Section 3.2.3. This involves choosing a fixed solution from the reference set $\mathcal{S}_{\text{Ref}}$ and combining it with other solutions from the entire set of diversified solutions $\mathcal{P}$. Another approach involves selecting a subset of solutions from the reference set $\mathcal{S}_{\text{Ref}}$ and combining them to either enhanced or diversified solutions.

The updating of the subsets is influenced by both the reference set $\mathcal{S}_{\text{Ref}}$, containing better solutions, and the diversified set having various solutions $\mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$. The reference set is consistently updated when new and improved solutions are attained or when new configurations with identical objective values are reached, while the diversified solutions replace some solutions belonging to $\mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$. Such a strategy aims to maintain both subsets throughout the search process. In this study, the diversity of the reference set is assessed based on the Hamming distance, where the diversity $\texttt{distance}(\mathcal{P})$ of the overall solutions, denoted as $\mathcal{P}$, is defined as follows:

$$\texttt{distance}(\mathcal{P}) = \max \left\{ \min \left\{ \texttt{distance}(S, S') \mid S' \in \mathcal{P} \right\} : S \in \mathcal{P} \right\}$$

The max-min measure, as successfully employed in [18], means that a solution is incorporated into $\mathcal{P}$ if it enhances the diversity of $\mathcal{P}$. This happens when the value of $\texttt{distance}(\mathcal{P})$ increases or if the minimum distance between the candidate and the other solutions in the reference set is greater than the current minimum distance between two solutions in the reference set. Throughout each step of the search process, only distinct solutions belonging to $\mathcal{P}$ are retained.

3.2.4. Subsets generation and combination methods

In SS, the reference set update method is specialized in selecting the best solutions generated during an optimization process and incorporating them into the reference set. This reference set acts as a storage for the best solutions identified thus far, guiding the search process towards the global optimum. In contrast, the combination method produces novel solutions by combining or modifying existing ones.

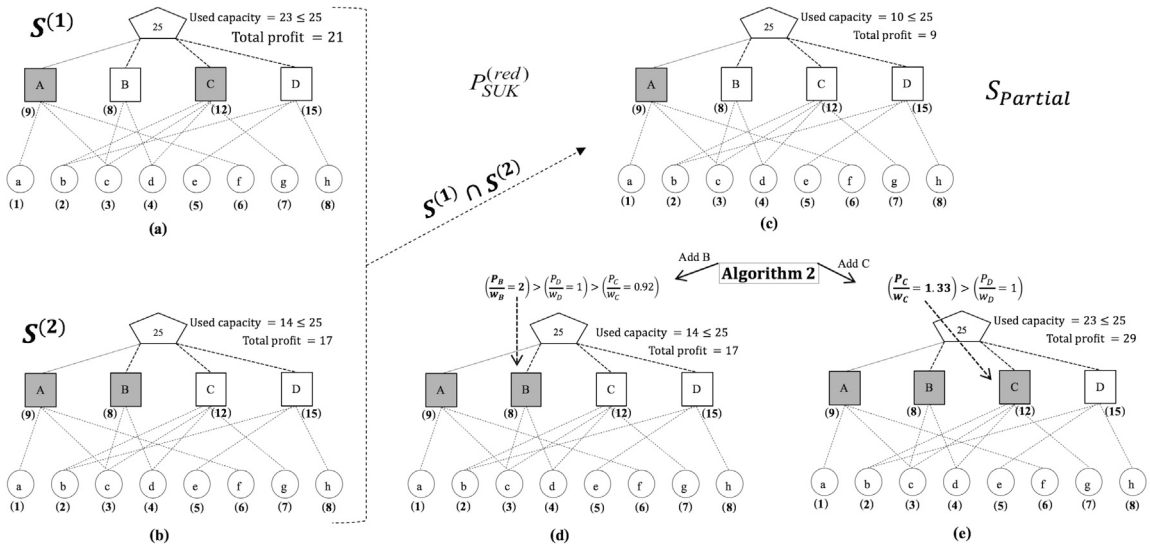


Fig. 3. The process illustrating the first level of the combination of two solutions and the generation of the new one.

These combinations may incorporate the crossover operator, similar to the one used in a standard genetic algorithm but in an extended form. Instead of relying on conventional operators like one-point, two-points, or others, the proposed method employs the fusion operator. Its goal is to create a new solution by combining diverse existing solutions from the reference set $\mathcal{S}_{\text{Ref}}$ and the diversified set $\mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$. More precisely, let $S^{(1)}$ belonging to the α-best solutions (which refer to a set of α selected solutions that are considered among the top performers based on the defined objective function values) of the current reference set $\mathcal{S}_{\text{Ref}}$, and $S^{(2)} \in \mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$ be the solutions being combined, and S_{new} be the resulting new solution built as follows:

1. Set $S_{Partial} = S^{(1)} \cap S^{(2)}$, where only items fixed to one in both solutions are stored.
2. Let $P^{(red)}_{SUK}$ be the reduced problem obtained by fixing all items of $S_{Partial}$ whose values are fixed to one.
3. Call `Algorithm 2` to optimize the reduced problem $P^{(red)}_{SUK}$ for completing the solution $S_{Partial}$, resulting in $S_{Partial}$ as the final solution: note that resulting solution as S_{new}.

Referring to the example depicted in Fig. 1.(i), we now explore the application of the proposed method. Fig. 3 illustrates the process of combining two given solutions. The first feasible solution, denoted as $S^{(1)} = \{1, 0, 1, 0\}$ (cf., Fig. 3.(a)), represents the selection of items A and C with $z(S^{(1)}) = p_A + p_C = 21$. The second feasible solution, $S^{(2)} = \{1, 1, 0, 0\}$ (cf., Fig. 3.(b)), denotes the selection of items A and B with $z(S^{(2)}) = p_A + p_B = 17$. Next, the first combining method applies an intersection strategy of both solutions $S^{(1)}$ and $S^{(2)}$ in order to select the items belonging to both solutions, where it yields a partial solution $S_{Partial} = \{1, 0, 0, 0\}$ (cf. Fig. 3.(c)).

Thus, Algorithm 2 is called to enhance (complete) the provided partial solution $S_{Partial}$. For this example, two steps are considered, where items are selected to be introduced into the incremental solution whenever they achieve a better profit-to-weight ratio while satisfying the capacity constraint. Hence, the following two iterations of the algorithm are considered:

1) `First iteration.` Let $S_{Partial} = \{1, 0, 0, 0\}$ represent a partial incomplete solution, $\bar{I} = \{B, C, D\} = \{0, 1, 1, 1\}$, and the capacity of the reduced subproblem P^{r}_{SUK} be $\bar{b} = b - 10 = 15$. According to Algorithm 2, all remaining items are sorted in decreasing order of their profit-to-weight ratio: $\frac{p_B}{w_B} = \frac{8}{4} = 2 > \frac{p_D}{w_D} = \frac{15}{15} = 1 > \frac{p_C}{w_C} = \frac{12}{14} = 0.92$. Following the greedy principle, item B with the maximum profit-per-weight ratio is added to $S_{Partial}$, resulting in $S_{new} = S_{Partial} \cup \{B\} = \{1, 1, 0, 0\}$ (cf., Fig. 3.(d)). The aggregate weight is $W(S_{new}) = w_a + w_c + w_d + w_f = 14$, and the objective value is $z(S_{new}) = p_A + p_B = 17$. The capacity of the reduced subproblem P^{r}_{SUK} becomes $\bar{b} = \bar{b} - w_d = 11$, $\bar{I} = \bar{I} \setminus \{B\} = \{C, D\} = \{0, 0, 1, 1\}$. As $\bar{I} \neq \emptyset$ and $W(S_{new}) < 25$, the second iteration is conducted.
2) `Second iteration.` For the remaining unassigned items in $\bar{I}$, they are sorted in decreasing order of their recalculated profit-to-weight ratio: $\frac{p_C}{w_C} = \frac{12}{9} = 1.33$, and $\frac{p_D}{w_D} = \frac{15}{15} = 1$. Therefore, item C is added to S_{new}, resulting in $S_{new} = \{A, B, C\} = \{1, 1, 1, 0\}$ (cf., Fig. 3.(e)), $\bar{I} = \bar{I} \setminus \{C\} = \{D\} = \{0, 0, 0, 1\}$. The aggregate weight is $W(S_{new}) = w_a + w_b + w_c + w_d + w_f + w_g = 23$, and the objective value is $z(S_{new}) = p_A + p_B + p_C = 29$. The capacity of the reduced subproblem P^{r}_{SUK} becomes $\bar{b} = \bar{b} - w_b - w_g = 2$. Since S_{new} no longer satisfies the capacity constraint when proceeding to the next iteration, the algorithm terminates.

Furthermore, whenever more than two solutions are considered, the k-th solution, namely $S^{(k)}$, can be combined with the resulting solution S_{new} using the above process. Thus, from the preceding steps, each subset associated with a selected solution from the α-best solutions (combined with the remaining solutions from $\mathcal{S}_{\text{Ref}}$) may be constructed using an iterative approach.

Algorithm 4 Adaptive Evolutionary Search-Based Method (AES-BM).

Input. A instance of P_{SUK}.
Output. A final resulting solution S_{Best}.

1: Apply the `generation method` using `Algorithm 2` for building the archive set $\mathcal{P}$ with its reference set $\mathcal{S}_{\text{Ref}}$ and complementary set $\mathcal{S}_{\text{Div}}$ (cf. Section 3.2.1).
2: **repeat**
3: Apply the `improvement method` (Algorithm 3) to the solutions of the reference set $\mathcal{S}_{\text{Ref}}$ (cf. Section 3.2.2).
4: Call the `reference update` and `combination methods` (which use the `subset generation method` (Section 3.2.3)):
(a) To create an expanded set $\mathcal{A}$ of solutions.
(b) To merge all solutions of $\mathcal{A}$ with those of $\mathcal{S}_{\text{Ref}}$.
(c) To enhance and diversify the solutions related to the reference set $\mathcal{S}_{\text{Ref}}$ and the complementary one $\mathcal{S}_{\text{Div}} = \mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$ (as highlighted in Section 3.2.4).
5: **until** (the stopping criteria is performed)
6: **Return** the resulting best solution $S_{\text{Best}} \in \mathcal{S}_{\text{Ref}}$.

Consequently, a sequence of subsets is generated; this is referred to as the *"subset generation method"*. Finally, for each solution ℓ belonging to the α-best solutions, a subset of solutions, namely $\mathcal{S}^{(\ell)}$, can be constructed by employing that method, and so the resulting set of solutions can be stated as $\mathcal{A} = \bigcup_{\ell=1}^{\alpha} \mathcal{S}^{(\ell)}$.

Thus, the reference update strategy, involving solutions susceptible to integrating the reference set and the diversified set, is employed to filter all solutions belonging to $\mathcal{A}$; those obtained through the combination method, as described above.

3.2.5. An overview of the adaptive evolutionary search-based method

Algorithm 4 summarizes the main steps of the proposed Adaptive Evolutionary Search-Based Method (AES-BM). Initially (line 1), the global set of diversified solutions $\mathcal{P}$ is generated using the filling strategy principle (Algorithm 2) that randomly repeated for creating the final set composed of both the reference set $\mathcal{S}_{\text{Ref}}$ and the rest of the diversified solutions (Section 3.2.1 for the starting set $\mathcal{P}$, and Section 3.2.3 how the diversified solutions are selected). Next, the loop "repeat" (line 4) iteratively explores the solutions of the reference set $\mathcal{S}_{\text{Ref}}$ that is combined with the other solutions. In this case, the "`improvement method`" (Algorithm 3) is first applied (line 3) to either enhance the quality of the solutions according to the objective function or diversify the solutions according to Hamming distance employed. Thus, the "`reference set update method`" hybridized to the "`combination method`" (cf., Section 3.2.4) is used to establish a series of subsets of solutions. In fact, here the "`subset generation method`" is called by the above hybridized method to expand the search space and obtain a series of subsets of solutions around all solutions of the current reference set $\mathcal{S}_{\text{Ref}}$, combined with the other solutions using the fusion operator (Section 3.2.4). Hence, the solutions that will be included in the current reference set $\mathcal{S}_{\text{Ref}}$ and those included in the created set $\mathcal{A}$ are merged. Finally, the algorithm stops when the runtime limit is attained. In this case, it exits with the best solution S_{Best} belonging to the reference set $\mathcal{S}_{\text{Ref}}$.

3.3. Computational complexity analysis

To analyze the computational complexity of Algorithm 4 (AES-BM), we need to evaluate the complexity of each step within the algorithm. AES-BM employs the tailored procedure (Section 3.2.1, Algorithm 2)) as its generation method. Its efficiency heavily depends on the sorting step, where items are arranged in non-ascending order based on their profit per weight ratio. The sorting operation typically needs a time complexity of $O(m \log m)$, where m represents the number of items in the problem instance; thus, the complexity of that procedure can be evaluated to $O(|\mathcal{P}| \times m \log m)$.

For the main loop of AES-BM, the computational complexity primarily relies on two key components: the improvement method and, the reference update and combination method. In the case of the improvement method, which is the multi-neighborhood tabu search (Section 3.2.2, Algorithm 3), each solution S can be represented using the selected items and the unselected items, denoted as $< S^{(1)}, \bar{S}^{(1)} >$. The complexity of the multi-neighborhood tabu search (MTS) procedure is $O((m + |S^{(1)}| \times |\bar{S}^{(1)}|) \times K)$, where K is a constant related to the number of iterations in the MTS procedure; thus, the complexity can be evaluated to $O((m + |S^{(1)}| \times |\bar{S}^{(1)}|))$. The computational complexity of the updating of subsets and combination methods is influenced by both the reference set $\mathcal{S}_{\text{Ref}}$ and the diversified set $\mathcal{P} \setminus \mathcal{S}_{\text{Ref}}$. The process involves evaluating the diversity of the overall solutions $\mathcal{P}$ using the Hamming distance measure. Calculating the diversity involves comparing each solution in $\mathcal{P}$ with every other solution, resulting in a time complexity of $O(|\mathcal{P}|)$. Overall, the time complexity of the approach is dominated by the combination generation step and the diversity assessment step, resulting in a combined complexity of $O((|\mathcal{S}_{\text{Ref}}| + 1) \times |\mathcal{P}|)$. In Algorithm 4, the main loop iterates until the stopping criteria are matched, with each iteration involving the improvement method and the combination methods. Hence, given that K and T are predefined constants, the final time complexity can be estimated as $O(m \log m + (m + |S^{(1)}| \times |\bar{S}^{(1)}|) + |\mathcal{P}|)$.

4. Computational results

In this section, the performance of the proposed Adaptive Evolutionary Search-Based Method (AES-BM) is evaluated on benchmark instances extracted from existing literature. These instances are categorized into two groups: a first group, noted `Group 1`, containing the so-called medium-sized instances and a second group, noted `Group 2`, including the so-called large-scale instances. On one hand, `Group 1`, extracted from [23] and provided using a random generator extracted from [13], includes three sets: `Set I`, `Set II`, and `Set III`. The first one (`Set I`) includes ten instances where the number of items is less than the number of elements, the second one (`Set II`) contains ten instances with a number of items equals to the number of elements, and the third one (`Set III`) has ten instances with the number of items greater than the number of elements. Across these sets, the number of items and elements varies

from 85 to 500. On the other hand, `Group 2`, taken from [4], includes three sets: `Sets IV, V`, and `VI`. The last group of instances represents larger-sized instances, where the number of items and elements ranges from 500 to 1000.

4.1. Experimental platform and parameter settings

The designed method AES-BM, along with its various versions, was implemented in C++ and initially executed in MatriCS[1] for optimizing the parameters used by the method. Subsequently, it ran on a PC IP Core i7 (3.6 GHz), and 32 GB of RAM, under Ubuntu 22.04 operating system. Furthermore, the considered instances were generated using the procedure designed in [23,24]. Finally, the AES-BM and the comparison algorithms were run on the same platform and using the same runtime limits. Specifically, for instances with $m \leq 500$ items, the runtime limit was fixed to 500 seconds, while for instances with $m > 500$ items, the runtime limit was extended to 1000 seconds. Each algorithm, including both `Group 1` and `Group 2`, was independently executed 10 times using different random seeds. The evaluation was based on the best bound and average results obtained across all test instances. For the comparison algorithms DHJaya(1,2) [26], IRSP [4], I2PLS [23], HSPO-BA [6], and PSO-B [5], their results, obtained under the same computing platform and stopping conditions as this study, were directly adopted from [5]. Regarding the KBTS algorithm[2] [24], we reran the code under the same platform and conditions to obtain corresponding results.

4.2. A better tuning for the reference set's size

In the preliminary study, the behavior of AES-BM parameters was tested on two sets of instances, each belonging to different groups: the first set (`Set III`) contains medium-sized instances, and the second one (`Set VI`) includes large-scale ones. The evaluation involved varying the size of the *reference set* $\mathcal{S}_{\text{Ref}}$, starting with a fixed minimum size of 5 and incrementing it by 10 for subsequent versions, and the size of the set containing all solutions $\mathcal{P}$ was fixed to five times than that of the reference set. The aforementioned choice was implemented after several tests, prioritizing the quality of the obtained solutions and aiming to prevent a significant increase in the runtime of the selected version.

Table 2 reports the results provided by AES-BM when varying the size of the reference and archive set. Column 1 of the table displays instance information, columns 2 to 4 display the best objective values achieved by the version with a population size of 25, the average bound over 5 trials, and the average runtime for this version. The next three columns display the same information for the second version with a population size of 50, while the last three columns tally results for the third version with a population size of 75. The following discussion analyzes the results presented in Table 2:

- For all considered population sizes, AES-BM with its all versions is able to provide good global average bounds, since the global values vary from 11813.82 (column 9, last line) to 11820.32 (column 6, last line).
- For overall instances, the AES-BM$_{50}$ version, with $|\mathcal{S}_{\text{Ref}}| = 10$, shows superior performance compared to other versions of the algorithm. Specifically, AES-BM$_{50}$ attains a global average value of 11820.32, resulting in a gap (AES-BM$_{50}$ - AES-BM$_{25}$, with $|\mathcal{S}_{\text{Ref}}| = 5$) of 2.16. When compared to the latest version, AES-BM$_{75}$, the positive gap (AES-BM$_{50}$ - AES-BM$_{75}$) maintains for the method with a population size of 50, totaling 6.5.
- AES-BM$_{50}$ successfully matches all the best bounds across the five groups of instances. Specifically, it achieves a matching rate of 47.67% compared to the best solutions, whereas AES-BM$_{75}$ achieves a slightly higher matching rate of 53.33%.

Fig. 4 illustrates the variation of the Gap, according to the best solution value, among the three versions of AES-BM on the instances of `Set III` and `Set VI`. From the curves presented in the figure, several non-negative gaps show that the second version is able to provide better results, even though the other two versions remain competitive.

According to the average runtimes required by the three versions of the algorithm, there is an observed increase in runtime with the growth of the population size, assuming no specific criterion has been set for runtime. In this case, the version with a population of 25 agents slightly outperforms the others, needing an average runtime of 180.73 seconds. The second version, with a population of 50 agents, needs 191.35 seconds, while the final version necessitates 199.05 seconds. Because we are interested with solution quality in this study, the second version with a population of 50 solutions will be retained for the remainder of the experimental part.

4.3. Behavior of the used improvement strategies

In order to assess the impact of the employed strategies in the proposed AES-BM, especially when applying the improvement method, four versions of AES-BM are considered. The first version, AES-BM$_{(1)}$, represents the algorithm without incorporating the tabu search strategy (both the simple tabu list and tabu tenure) such that only the adding and the moving operators are employed (as described in Section 3.2.2, equations (8) and (9), respectively). The second version, AES-BM$_{(2)}$, operates without employing tabu search (both the simple tabu list and tabu tenure), while the initial population is built using the random generator, almost of using the proposed tailored generator (cf., Section 3.2.1). The third version, AES-BM$_{(3)}$, represents the algorithm using the tailored procedure for generating the starting population, the simple tabu strategy and without introducing tabu tenure. Finally the last

[1] https://www.matrics.u-picardie.fr.

[2] The code is available on the website provided by the authors for further research.

Table 2
Behavior of AES-BM when varying the reference and archive set size.

	$\|S_{Ref}\|=5$, $\|P\|=25$			$\|S_{Ref}\|=10$, $\|P\|=50$			$\|S_{Ref}\|=15$, $\|P\|=75$		
#Inst	Best	Av. Best	Av. Time	Best	Av. Best	Av. Time	Best	Av. Best	Av. Time
S-III-1	13283	13283.00	5.64	13283	13283.00	9.84	13283	13283.00	3.95
S-III-2	12479	12479.00	263.70	12479	12479.00	227.39	12479	12479.00	163.23
S-III-3	13521	13521.00	6.28	13521	13521.00	7.45	13521	13521.00	4.32
S-III-4	14215	14092.00	132.78	14215	13995.20	64.46	14044	14013.40	99.78
S-III-5	11563	11563.00	13.14	11563	11563.00	4.28	11563	11563.00	12.96
S-III-6	12607	12450.20	50.38	12607	12468.00	138.55	12607	12450.20	179.86
S-III-7	11484	11484.00	3.61	11484	11484.00	3.90	11484	11484.00	3.21
S-III-8	11209	11209.00	13.57	11209	11209.00	63.24	11209	11209.00	135.60
S-III-9	11771	11771.00	87.90	11771	11771.00	141.74	11771	11771.00	78.39
S-III-10	10217	10207.80	69.26	10238	10216.20	195.54	10238	10221.20	141.61
Average	12234.90	12206.00	64.63	**12237.00**	**12198.94**	85.64	12219.90	12199.48	82.29
	$\|S_{Ref}\|=5$, $\|P\|=25$			$\|S_{Ref}\|=10$, $\|P\|=50$			$\|S_{Ref}\|=15$, $\|P\|=75$		
#Inst	Best	Av. Best	Av. Time	Best	Av. Best	Av. Time	Best	Av. Best	Av. Time
S-VI-1	13044	12890.40	258.66	13044	12941.60	545.80	13044	12992.80	458.62
S-VI-2	7336	7153.00	383.51	7454	7269.60	337.86	7363	7163.80	544.60
S-VI-3	15505	15480.60	369.68	15505	15460.60	260.60	15505	15460.60	293.55
S-VI-4	8765	8765.00	130.23	8765	8765.00	296.28	8765	8765.00	239.79
S-VI-5	18959	18948.20	533.96	18994	18930.00	326.69	18994	18952.00	373.36
S-VI-6	10848	10831.00	308.72	10848	10848.00	413.30	10848	10840.60	437.16
S-VI-7	11853	11767.40	442.03	11853	11681.80	263.95	11853	11589.60	329.60
S-VI-8	6901	6687.40	230.33	6901	6588.20	414.01	6882	6683.60	342.17
S-VI-9	14186	14186.00	58.84	14186	14186.00	115.86	14186	14186.00	104.12
S-VI-10	8196	8196.00	140.80	8196	8196.00	100.53	8196	8196.00	235.36
S-VI-11	17667	17620.60	297.50	17667	17632.20	255.09	17667	17632.20	405.25
S-VI-12	9920	9804.20	742.62	9920	9816.00	477.21	9920	9878.40	464.43
S-VI-13	11217	10714.20	273.37	11217	10765.00	277.05	10963	10567.40	146.38
S-VI-14	6763	6763.00	6.05	6763	6763.00	8.71	6763	6763.00	7.93
S-VI-15	12983	12871.40	360.04	13122	12995.20	361.79	13122	12986.80	475.64
S-VI-16	7683	7683.00	15.99	7683	7683.00	11.96	7683	7683.00	12.56
S-VI-17	16044	15944.40	401.49	16044	15988.00	460.17	16044	15923.00	325.12
S-VI-18	9443	9440.00	389.16	9443	9441.40	420.07	9443	9443.00	489.10
Average	11517.39	11430.32	296.83	**11533.61**	**11441.70**	297.05	11513.39	11428.16	315.82
Global Av	11876.14	11818.16	180.73	**11885.31**	**11820.32**	191.35	11866.64	11813.82	199.05

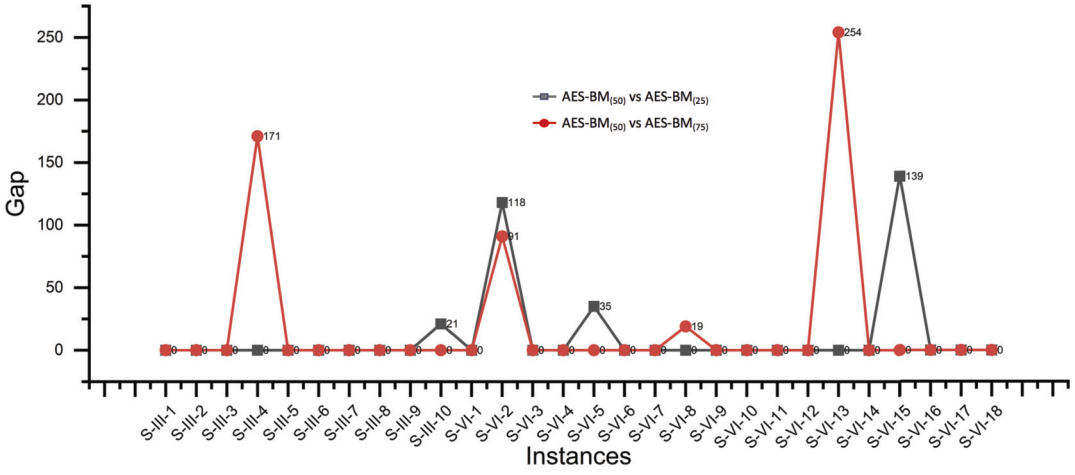


Fig. 4. Variations of the Gap achieved by AES-BM with a population size of versus the two other versions of the algorithm.

version, AES-BM$_{(Full)}$, denotes the full version of the algorithm, where all strategies are combined. These versions of AES-BM allow for a comprehensive analysis of the algorithm's performance according to different configurations and initialization strategies.

The results provided by the four versions of the algorithm, i.e., AES-BM$_{(1)}$, AES-BM$_{(2)}$, AES-BM$_{(3)}$, and AES-BM$_{(Full)}$, are reported in Table 3 for the first group of instances (`Set I`, `Set II`, and `Set III`), and Table 4 for instances of the second group (`Set IV`, `Set V`, and `Set VI`). Column 1 (of each table) displays the instance label. Columns from 2 to 5 report the best objective value

Table 3
Effect of the strategies used by the proposed AES-BM: `Group 1`.

	Various versions of AES-BM				AES-BM$_{(Full)}$) vs other versions		
#Inst	AES-BM$_{(1)}$	AES-BM$_{(2)}$	AES-BM$_{(3)}$	AES-BM$_{(Full)}$)	Gap's Variation		
S-I-1	12045	12045	12045	12045	0	0	0
S-I-2	12369	12369	12369	12369	0	0	0
S-I-3	11739	12576	13696	13696	1957	1120	0
S-I-4	10463	10067	11298	11298	835	1231	0
S-I-5	9998	10540	11568	11568	1570	1028	0
S-I-6	9647	10252	11763	11802	2155	1550	39
S-I-7	8672	9330	10421	10600	1928	1270	179
S-I-8	8224	8371	10506	10506	2282	2135	0
S-I-9	9017	9722	11009	11321	2304	1599	312
S-I-10	7613	8281	10117	10220	2607	1939	103
S-II-1	13937	13937	14044	14044	107	107	0
S-II-2	13508	13508	13498	13508	0	0	10
S-II-3	10446	10468	12522	12522	2076	2054	0
S-II-4	11210	11365	12317	12317	1107	952	0
S-II-5	10421	11462	12736	12817	2396	1355	81
S-II-6	9609	9769	11425	11585	1976	1816	160
S-II-7	9508	9853	11665	11665	2157	1812	0
S-II-8	8385	8539	11325	11325	2940	2786	0
S-II-9	9077	9710	11123	11249	2172	1539	126
S-II-10	8076	8752	10186	10381	2305	1629	195
S-III-1	13125	13089	13283	13283	158	194	0
S-III-2	12479	12479	12479	12479	0	0	0
S-III-3	11522	12154	13466	13521	1999	1367	55
S-III-4	12724	12495	14187	14215	1491	1720	28
S-III-5	9178	9795	11498	11563	2385	1768	65
S-III-6	9549	10023	12339	12607	3058	2584	268
S-III-7	9416	10163	11484	11484	2068	1321	0
S-III-8	8106	8775	11209	11209	3103	2434	0
S-III-9	8880	10333	11671	11771	2891	1438	100
S-III-10	7971	8231	10007	10238	2267	2007	231
Average	10230.47	10615.10	11908.53	11973.60	1743.13	1358.50	65.07

achieved by the four versions of AES-BM. The last three columns tally the gap between the objective values reached by the full version AES-BM$_{(Full)}$ and the other versions of the algorithm, i.e., AES-BM$_{(Full)}$− AES-BM$_{(1)}$, AES-BM$_{(Full)}$− AES-BM$_{(2)}$, and AES-BM$_{(Full)}$− AES-BM$_{(3)}$, respectively. The last line of both tables displays the average value related to each version of the algorithm AES-BM and those related to the aforementioned computed gaps.

In what follows, we comment on the results of both Tables 3 and 4. In the case of the first group of instances (Table 3):

- It is observed that AES-BM$_{(2)}$ outperforms the first version AES-BM$_{(1)}$. This is supported by (i) the higher average achieved by the second version, reaching 10615.10 compared to 10230.47 for the first version, and (ii) the frequency with which the second version consistently provides solutions of superior quality (22 instances with better solution values and 5 instances of equality). In this case, building the initial population through a specific procedure is understood as resulting in final solutions of higher quality.
- AES-BM$_{(3)}$ displays superior performance compared to both AES-BM$_{(1)}$ and AES-BM$_{(2)}$. Specifically, this version attains an average result of 11908.53, surpassing the results of the other two versions (10230.47 for AES-BM$_{(1)}$ and 10615.10 for AES-BM$_{(2)}$). Moreover, there are improvements in 26 instances, with equality observed in 3 instances. The obtained results illustrate the effect of introducing a simple tabu search when combined with the specific procedure used for generating the starting population.
- Finally, it can be observed that the full version, AES-BM$_{(Full)}$, displays better results when these different strategies are combined in a unified manner. Indeed, this version displays improvements in 15 occasions and matches the best solutions achieved by the first three versions on 15 occasions.

In the case of the second group of instances (Table 4), the same phenomenon, as observed in the first group of instances, can be noted. Indeed, among the first three versions of the algorithm, it is observed that the quality of solutions produced by AES-BM$_{(1)}$ (i.e., 9474.65) remains inferior to those produced by AES-BM$_{(2)}$ (i.e., 10349.81), and the AES-BM$_{(3)}$ version yields better results (i.e., 12045.96). Finally, the full version AES-BM$_{(Full)}$ performs better, achieving an average result of 12045.96, when the different strategies are combined. In this case, one can observe that the Gap varies from 421.61 (between AES-BM$_{(Full)}$ and AES-BM$_{(3)}$) and 2571.31 (between AES-BM$_{(Full)}$ and AES-BM$_{(1)}$).

Fig. 5 shows the variation of the Gap between AES-BM$_{(Full)}$ versus the three first versions AES-BM$_{(1)}$, AES-BM$_{(2)}$ and AES-BM$_{(3)}$, respectively. The x-axis represents the index of the overall instances belonging to both groups, i.e., `Set I`, `Set II`, `Set III`, `Set`

Table 4
Effect of the strategies used by the proposed AES-BM: `Group 2`.

	Various versions of AES-BM				AES-BM$_{(Full)}$) vs other versions		
#Inst	AES-BM$_{(1)}$	AES-BM$_{(2)}$	AES-BM$_{(3)}$	AES-BM$_{(Full)}$)	Gap's Variation		
S-IV-1	10270	11420	12732	13170	2900	1750	438
S-IV-2	6734	6990	7647	7962	1228	972	315
S-IV-3	11338	13519	15428	15762	4424	2243	334
S-IV-4	7811	8358	9328	9423	1612	1065	95
S-IV-5	13267	17249	19697	20221	6954	2972	524
S-IV-6	9243	10217	11450	11730	2487	1513	280
S-IV-7	9766	11410	13966	14211	4445	2801	245
S-IV-8	6473	6731	7521	7801	1328	1070	280
S-IV-9	11486	13502	17175	17364	5878	3862	189
S-IV-10	7234	7995	9166	9385	2151	1390	219
S-IV-11	13606	17926	21364	22058	8452	4132	694
S-IV-12	8628	9648	11322	11818	3190	2170	496
S-IV-13	10253	11674	13820	14296	4043	2622	476
S-IV-14	6932	6942	8125	8327	1395	1385	202
S-IV-15	11645	14288	17063	17752	6107	3464	689
S-IV-16	7591	8296	9557	9917	2326	1621	360
S-IV-17	13744	17448	22170	22854	9110	5406	684
S-IV-18	9243	10025	12069	12361	3118	2336	292
S-V-1	10093	10567	11705	12135	2042	1568	430
S-V-2	6535	6530	7043	7070	535	540	27
S-V-3	11586	12240	14065	14703	3117	2463	638
S-V-4	7584	7547	8305	8410	826	863	105
S-V-5	13700	15717	17211	18270	4570	2553	1059
S-V-6	8694	8895	10020	10250	1556	1355	230
S-V-7	9698	9919	10649	11357	1659	1438	708
S-V-8	6515	6308	6818	6818	303	510	0
S-V-9	11408	11606	12594	13381	1973	1775	787
S-V-10	7245	7143	7901	7940	695	797	39
S-V-11	13256	14116	15395	16316	3060	2200	921
S-V-12	8475	8511	9437	9684	1209	1173	247
S-V-13	9194	9382	10182	10535	1341	1153	353
S-V-14	5697	5963	6154	6190	493	227	36
S-V-15	10558	10907	11653	12483	1925	1576	830
S-V-16	6808	6818	6980	7428	620	610	448
S-V-17	12521	12672	14163	14965	2444	2293	802
S-V-18	7767	7891	8701	8773	1006	882	72
S-VI-1	10168	11281	12420	13044	2876	1763	624
S-VI-2	6268	6727	7031	7454	1186	727	423
S-VI-3	12135	12884	15026	15505	3370	2621	479
S-VI-4	7198	7446	8382	8765	1567	1319	383
S-VI-5	14602	16068	18840	18898	4296	2830	58
S-VI-6	8546	9271	10510	10848	2302	1577	338
S-VI-7	9462	10711	11165	11853	2391	1142	688
S-VI-8	5904	6423	6850	6901	997	478	51
S-VI-9	11392	12782	13426	14156	2764	1374	730
S-VI-10	6541	7393	7723	8196	1655	803	473
S-VI-11	13169	14638	16083	17667	4498	3029	1584
S-VI-12	7901	8711	9110	9920	2019	1209	810
S-VI-13	9747	10534	10961	11217	1470	683	256
S-VI-14	6129	6116	6376	6763	634	647	387
S-VI-15	11182	11765	12957	13122	1940	1357	165
S-VI-16	7107	7200	7683	7683	576	483	0
S-VI-17	13105	14043	15511	15935	2830	1892	424
S-VI-18	8477	8527	9085	9435	958	908	350
Average	9474.65	10349.81	11624.35	12045.96	2571.31	1696.15	421.61

`IV`, `Set V`, and `Set VI`. As shown from the figure, it confirms a good behavior of AES-BM$_{(Full)}$, which is followed by the third version AES-BM$_{(3)}$.

To attain solutions of superior quality, characterized by the highest objective values and enhanced bounds, we opt to consistently apply the full version of the algorithm for all subsequent computational results.

Table 5
Comparison of average objective values provided by AES-BM with various available methods.

Group 1	#Inst	DHJaya(1.2)	I2PLS	KBTS	HSPO-BA	PSO-B	This work
	S-I-1	11570.6	12045	12045	12045	12045	12045
	S-I-2	12318	12315.53	12369	12369	12369	12369
	S-I-3	13350.2	13695.6	13696	13696	13696	13696
	S-I-4	10828.9	11276.17	11298	11298	11298	11298
	S-I-5	11327.7	11568	11568	11565	11568	11568
	S-I-6	11025.9	11790.43	11799.27	11802	11802	11802
	S-I-7	10017	10536.53	10600	10597	10598	10600
	S-I-8	9565.72	10502.64	10506	10506	10506	10506
	S-I-9	10754.8	11306.47	11318.81	11313.8	11317.4	**11321**
	S-I-10	9467.8	10179.45	10219.76	10217.6	10218.8	**10220**
	S-II-1	13912.5	14044	14044	14044	14044	14044
	S-II-2	13439.1	13451.5	13508	13508	13508	13508
	S-II-3	12171.6	12522	12522	12522	12522	12522
	S-II-4	11746	12280.07	12317	12292	12303.3	**12317**
	S-II-5	12569.3	12817	12817	12817	12817	12817
	S-II-6	10701.9	11512.18	11584.17	11535.5	11576.2	**11585**
	S-II-7	10914.8	11665	11665	11665	11662.1	11665
	S-II-8	9969.9	11325	11325	11325	11325	11325
	S-II-9	10703.5	11243.4	11248.96	11249	11249	11249
	S-II-10	9801.5	10293.89	10362.63	10330.4	10376.2	**10381**
	S-III-1	13076	13283	13283	13283	13283	13283
	S-III-2	12192.5	12335.13	12479	12479	12479	12479
	S-III-3	13306.6	13521	13521	13521	13521	13521
	S-III-4	13660.2	14031.28	14209.87	14154.8	14180.4	14177
	S-III-5	10703.2	11562.02	11563	11563	11563	11563
	S-III-6	12037.5	12364.55	12536.02	12512.4	12542.7	12537.7
	S-III-7	11062	11484	11484	11484	11484	11484
	S-III-8	10017.9	11157.26	11209	11204.2	11206.6	**11209**
	S-III-9	11269.4	11729.76	11755.47	11763.5	11767.9	**11771**
	S-III-10	9354.28	10133.94	10202.9	10233.6	10234.6	**10235.7**
	Average	11427.88	11932.39	11968.56	11963.19	11968.77	**11969.95**
Group 2	#Inst	IRSP	I2PLS	KBTS	HSPO-BA	PSO-B	**This work**
	S-IV-1	12864.9	12650.4	13093.5	12994.8	13103	13122.9
	S-IV-2	7647	7581.4	7776.4	7647	7705.2	7779.2
	S-IV-3	15708.4	15464	15739.1	15752.6	15760	15761.2
	S-IV-4	9321.4	9365	9417.4	9406.2	9411.8	9423
	S-IV-5	19724.5	19597	19979.8	20221	20221	20221
	S-IV-6	11685.6	11697.6	11698	11698	11698	11719.4
	S-IV-7	13759.4	13686.8	14113	14181.6	14191.4	14201.4
	S-IV-8	7501.7	7142	7756.6	7737.7	7739.2	7761.5
	S-IV-9	17227.7	17125.8	17236.9	17232	17258.4	17284.8
	S-IV-10	9140.7	8851.2	9293.6	9216.8	9270.6	9299.9
	S-IV-11	21555.8	21735.2	21783.5	21594	21815.4	21915.6
	S-IV-12	11322	11253	11669.6	11583.8	11655.9	11671.1
	S-IV-13	13969.6	13820.2	14237.2	14209.8	14229.8	14238.8
	S-IV-14	7880.8	7645	8068	8090.8	8102.2	8134.4
	S-IV-15	17348.8	16890	17647.8	17456.5	17639.1	17709.6
	S-IV-16	9442.8	8935	9851.8	9862.8	9892.6	9917
	S-IV-17	22047.7	21600	22471.7	22069.7	22483	22535.2
	S-IV-18	12069	11557	12330	12222	12264.9	12339.3
	S-V-1	11831.1	11536.4	12122	11803.6	12053.2	12124.6
	S-V-2	6927.4	6553	7043	7031.4	7039	7053.3
	S-V-3	14380.6	14298	14534.9	14487.1	14652.3	14654
	S-V-4	8347	8347	8347	8347	8347	8366.3
	S-V-5	18027.7	17914	17969.8	17925	18111.7	18157.8
	S-V-6	10145	10145	10187.6	10198.4	10207.3	10217.2
	S-V-7	10782.3	10770	11129	11180.2	11256.8	11285.7
	S-V-8	6818	6713	6818	6789.1	6818	6818
	S-V-9	12401.3	12047	13297.7	12798.5	13122.2	13330.1
	S-V-10	7937.5	7712	7734.6	7934.5	7938	7892.6
	S-V-11	15403.6	14962	16238	15438.4	16197	16248.9
	S-V-12	9072	9437	9461.7	9626.2	9669.5	9686.4
	S-V-13	10381.9	9741	10530.8	10239.9	10450.1	10535
	S-V-14	6169.8	5698	6189.2	6169.1	6235	6190

(*continued on next page*)

Table 5 (*continued*)

Group 2	#Inst	IRSP	I2PLS	KBTS	HSPO-BA	PSO-B	**This work**
	S-V-15	12056	12049	12086	12057.3	12122.3	12145.5
	S-V-16	6979.4	6977	7428	7355.6	7378.2	7428.3
	S-V-17	14673.4	14546	14762.4	14701.1	14812.1	14845.9
	S-V-18	8501.8	8316	8764.2	8709.9	8742.4	8768.6
	S-VI-1	12641.1	12308	12788	12516.5	12649.5	12892.5
	S-VI-2	7031	7031	7353	7012.8	7242.6	7361.4
	S-VI-3	15383	15383	15383	15383	15383	15468.4
	S-VI-4	8735.6	8723	8765	8723	8739.8	8765
	S-VI-5	18885.5	18847	18867.5	18885.5	18940.2	18956.7
	S-VI-6	10328.6	10272	10817.7	10609.6	10740.5	10820.8
	S-VI-7	11153.3	10873	11639	11388.2	11544	11777.6
	S-VI-8	6780.4	6386	6881.1	6871.4	6873.7	6884.3
	S-VI-9	14012.9	13689	14125	13729	14045.1	14162.5
	S-VI-10	7723	7149	8196	8054.9	8137.5	8196
	S-VI-11	17462.7	17113	17530.8	17238.7	17544.7	17620.6
	S-VI-12	9481.8	9109	9663.4	9555.9	9642.8	9809.1
	S-VI-13	10963	10963	11181.1	10963	11081.4	11193.7
	S-VI-14	6376	6376	6763	6658.6	6694.5	6763
	S-VI-15	12957	12503	12573	12735.7	12960.4	12992.6
	S-VI-16	7347.6	7124	7683	7602.1	7630.6	7683
	S-VI-17	15764.8	15141	15941.9	15337.4	15835.3	15947.9
	S-VI-18	9020.7	8517	9414.3	9051.6	9230.7	9441.5
	Average	11724.10	11516.02	11932.84	11820.12	11935.37	**11991.11**

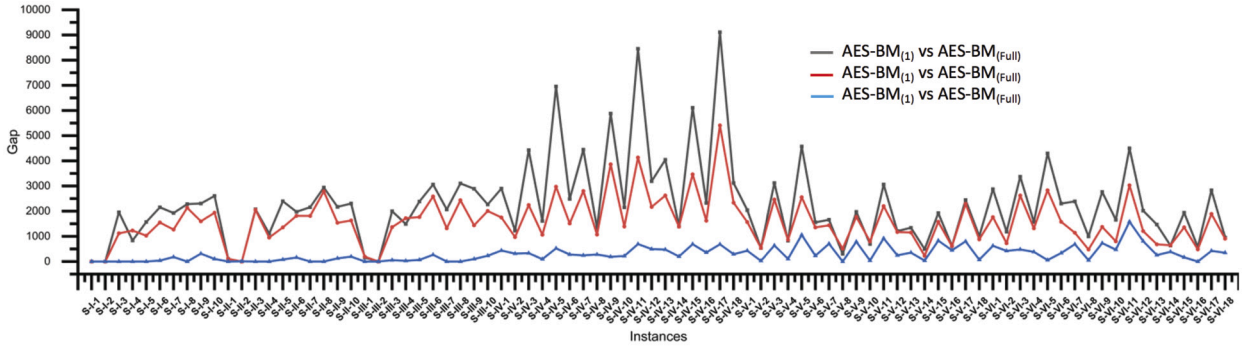


Fig. 5. Illustration of the variations of the different Gaps achieved by AES-BM$_{(Full)}$ versus the other version of the algorithm: both groups of instances.

4.4. Study of AES-BM's performance compared to recent methods

In order to assess the effectiveness of AES-BM (the full version), a comparison was conducted against state-of-the-art methods available in the literature, including DHJaya(1,2) [26], IRSP [4], I2PLS [23], KBTS [24], HSPO-BA [6], and PSO-B [5].

Table 5 (resp. Table 6) reports the average (best) objective values provided by all tested methods, which are evaluated on the overall instances of both groups (`Group 1` and `Group 2`), as used in [4,5]. The first two columns of each table display the group and the included instance with its label, while the rest of the columns tally the results related to global average objective values, over the ten trials, of all tested methods: DHJaya(1,2), IRSP, I2PLS, KBTS, HSPO-BA, PSO-B, and the proposed AES-BM (under "This work"), respectively. From Tables 5 and 6, the following observations are provided, as considered in [4,5].

4.4.1. Comment on instances of `Group 1`

By following the same test protocol as employed in [4,5], we provide an analysis of the results presented in the first parts (blocks) of both Tables 5 and 6. Specifically, in instances of `Group 1`, the following observations are made.

When comparing AES-BM to DHJaya(1,2), AES-BM outperforms DHJaya(1,2) by achieving an average global best solution value of 11439.40 (Table 6). This outperformance is evident as AES-BM outperforms DHJaya(1,2) by 534.20. In this case, the superiority extends to the global average value, reflecting the previously mentioned trend. The gap in this case reaches 542.07 over the ten trials considered.

In the comparison between AES-BM and I2PLS, AES-BM matches all the best bounds (Table 6) reported in the literature, particularly those achieved by I2PLS. On the other hand, the average results achieved by AES-BM are greater than those attained by I2PLS (11969.95 for AES-BM compared to 11932.39 achieved by I2PLS).

Table 6
Comparison of best objective values provided by AES-BM with various available methods on instances of both groupes: `Group 1` and `Group 2`.

Group 1	#Inst	DHJaya(1.2)	I2PLS	KBTS	HSPO-BA	PSO-B	This work
	S-I-1	12045	12045	12045	12045	12045	12045
	S-I-2	12369	12369	12369	12369	12369	12369
	S-I-3	13696	13696	13696	13696	13696	13696
	S-I-4	11298	11298	11298	11298	11298	11298
	S-I-5	11568	11568	11568	11568	11568	11568
	S-I-6	11401	11802	11802	11802	11802	11802
	S-I-7	10414	10600	10600	10600	10600	10600
	S-I-8	10302	10506	10506	10506	10506	10506
	S-I-9	10971	11321	11321	11321	11321	11321
	S-I-10	9715	10220	10220	10220	10220	10220
	S-II-1	14044	14044	14044	14044	14044	14044
	S-II-2	13508	13508	13508	13508	13508	13508
	S-II-3	12522	12522	12522	12522	12522	12522
	S-II-4	12187	12317	12317	12317	12317	12317
	S-II-5	12695	12817	12817	12817	12817	12817
	S-II-6	11113	11585	11585	11585	11585	11585
	S-II-7	11531	11665	11665	11665	11665	11665
	S-II-8	10915	11325	11325	11325	11325	11325
	S-II-9	1096	11249	11249	11249	11249	11249
	S-II-10	10176	10381	10381	10381	10381	10381
	S-III-1	13283	13283	13283	13283	13283	13283
	S-III-2	12274	12479	12479	12479	12479	12479
	S-III-3	13405	13521	13521	13521	13521	13521
	S-III-4	14215	14215	14215	14215	14215	14215
	S-III-5	10934	11563	11563	11563	11563	11563
	S-III-6	12245	12607	12607	12607	12607	12607
	S-III-7	11337	11484	11484	11484	11484	11484
	S-III-8	10431	11209	11209	11209	11209	11209
	S-III-9	11722	11771	11771	11771	11771	11771
	S-III-10	9770	10238	10217	10238	10238	10238
	Average	11439.40	11973.60	11972.90	**11973.60**	**11973.60**	**11973.60**
Group 2	#Inst	IRSP	I2PLS	KBTS	HSPO-BA	PSO-B	**This work**
	S-IV-1	12875	12774	13170	13170	13170	13170
	S-IV-2	7647	7565	7730	7647	7962	7962
	S-IV-3	15712	15464	15760	15760	15760	**15762**
	S-IV-4	9256	9365	9423	9423	9423	9423
	S-IV-5	19806	19597	20118	20221	20221	20221
	S-IV-6	11698	11698	11698	11698	11698	**11730**
	S-IV-7	13841	13841	14113	14211	14211	14211
	S-IV-8	7616	7142	7759	7759	7759	**7801**
	S-IV-9	17239	17175	17239	17232	17364	17364
	S-IV-10	9230	8939	9371	9230	9371	**9385**
	S-IV-11	21517	21748	22035	21601	22049	**22058**
	S-IV-12	11322	11322	11696	11696	11696	**11818**
	S-IV-13	14127	13903	14282	14296	14296	14296
	S-IV-14	8125	7647	8125	8125	8125	**8327**
	S-IV-15	17647	16912	17760	17541	17752	17760
	S-IV-16	9558	8935	9917	9917	9917	9917
	S-IV-17	22242	21600	**23004**	22355	**23004**	22854
	S-IV-18	12069	11557	12330	12330	12330	**12361**
	S-V-1	11866	11601	12122	12122	12122	**12135**
	S-V-2	6969	6553	7043	7043	7043	**7097**
	S-V-3	14442	14298	14698	14698	14703	14703
	S-V-4	8347	8347	8347	8347	8347	**8414**
	S-V-5	18164	17914	18138	17936	18197	**18270**
	S-V-6	10145	10145	10189	10234	10234	**10250**
	S-V-7	10799	10770	11143	11357	11357	11357
	S-V-8	6818	6713	6818	6790	6818	6818
	S-V-9	12407	12047	13355	13108	13355	**13381**
	S-V-10	7938	7712	7938	7938	7938	**7940**
	S-V-11	15679	14962	16316	16316	16316	16316
	S-V-12	9072	9437	9469	9659	9684	**9697**
	S-V-13	10485	9741	10535	10485	10535	10535
	S-V-14	6182	5698	6190	6182	**6332**	6190

(*continued on next page*)

Table 6 (*continued*)

Group 2	#Inst	IRSP	I2PLS	KBTS	HSPO-BA	PSO-B	**This work**
	S-V-15	12059	12049	12086	12086	12483	12483
	S-V-16	6980	6977	7428	7428	7428	**7429**
	S-V-17	14714	14546	14892	14879	14892	**14965**
	S-V-18	8746	8316	8773	8762	8773	8773
	S-VI-1	12742	12308	12788	12788	12788	**13044**
	S-VI-2	7031	7031	7454	7031	7454	7454
	S-VI-3	15383	15383	15383	15383	15383	**15505**
	S-VI-4	8765	8723	8765	8723	8765	8765
	S-VI-5	18893	18847	18868	18893	18994	18994
	S-VI-6	10318	10272	10848	10714	10848	10848
	S-VI-7	11218	10873	11639	11639	11853	11853
	S-VI-8	6879	6386	6882	6882	6882	**6901**
	S-VI-9	14070	13689	14125	14070	14125	**14186**
	S-VI-10	7723	7149	8196	8061	8196	8196
	S-VI-11	17532	17113	17609	17532	17667	17667
	S-VI-12	9575	9109	9808	9642	9798	**9920**
	S-VI-13	10963	10963	11203	10963	11203	**11217**
	S-VI-14	6376	6376	6763	6671	6763	6763
	S-VI-15	12957	12503	12573	12957	12983	**13122**
	S-VI-16	7683	7124	7683	7683	7683	7683
	S-VI-17	15818	15141	15989	15862	15913	**16044**
	S-VI-18	9088	8517	9443	9428	9428	**9443**
	Average	11784.31	11528.09	11982.02	11935.26	12025.76	**12051.44**

Turning to the comparison with KBTS, an enhanced version of I2PLS, one can expect some improvement in the quality of the solutions. Indeed, in this case, AES-BM maintains superiority in the average best solution value by 0.7 (Table 6), even though the global value over the ten trials slightly favors KBTS (1.38).

Finally, in the comparison of AES-BM to both HSPO-BA and PSO-B, it is observed that AES-BM, HSPO-BA, and PSO-B are competitive methods, especially on instances of `Group 1` containing medium-sized examples. All three methods achieve nearly the best and average bounds in the literature (the first part of both Table 5 and Table 6), with a slight advantage for HSPO-BA. The average value for these instances is 11963.19 for HSPO-BA, while it is 11968.77 for PSO-B and 11969.95 for AES-BM, respectively.

4.4.2. Comment on instances of `Group 2`

Following the same test protocol as employed previously (cf. Section 4.4.1), we provide an analysis of the results from this study presented in the second parts of both Tables 5 and 6.

In the comparison between AES-BM, IRSP, and I2PLS, AES-BM outperforms both IRSP and I2PLS (with solution values extracted from [4]). Across all instances belonging to `Group 2`, AES-BM provides superior solution values compared to those achieved by both IRSP and I2PLS. It is noteworthy that AES-BM dominates I2PLS, achieving a global best average value of 12051.44 compared to 11528.09 of I2PLS. Through comprehensive analysis across ten trials, AES-BM consistently demonstrates a superior average (lower) threshold, reaching 11991.11 compared to 11516.02.

In the comparison with KBTS, AES-BM also outperforms by achieving an average best global value of 12051.44, which is greater than that achieved by KBTS (11982.02). The positive gap related to the average global value, over the ten trials, is 58.27, expressing the stability of the method for these instances.

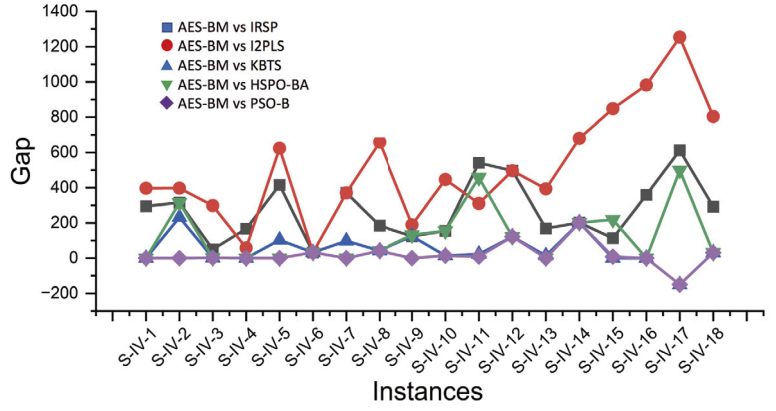
Comparing AES-BM to IRSP, AES-BM achieves a gap of 267.13 related to the difference between both average global best values. Additionally, the gap related to both average global bounds is globally the same, i.e., 267.01. In the comparison with HSPO-BA, AES-BM becomes highly competitive. The average global value related to the best bounds achieved is 12051.44 for AES-BM, while HSPO-BA achieves a value of 11935.26, resulting in a gap of 116.19.

Regarding the comparison with PSO-B, AES-BM appears competitive. The gap achieved by AES-BM related to the best bounds is 25.69, and it is 55.74 when comparing the average bounds over the ten trials.

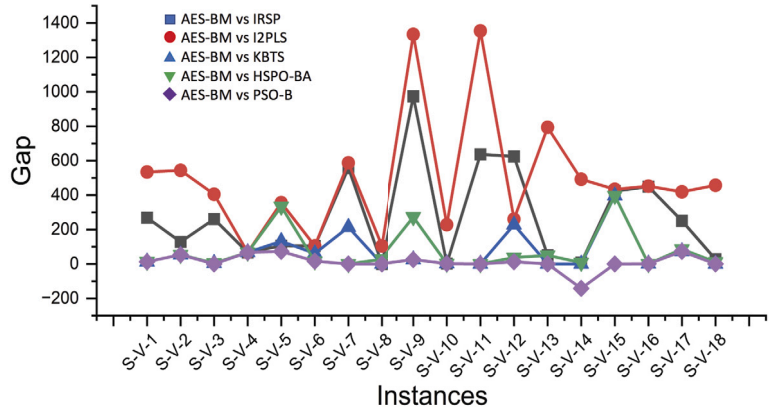
Finally, Fig. 6 illustrates the variation of the Gap (`Group 2`), according to the best solution value, between the full version of AES-BM and the five best methods. According to the curves displayed, one can observe the superiority of AES-BM when compared to both IRSP and I2PLS, while it performs slightly better than KBTS and remains competitive when compared to the two methods HSPO-BA and PSO-B.

4.5. Statistical analysis on the large-sized instances

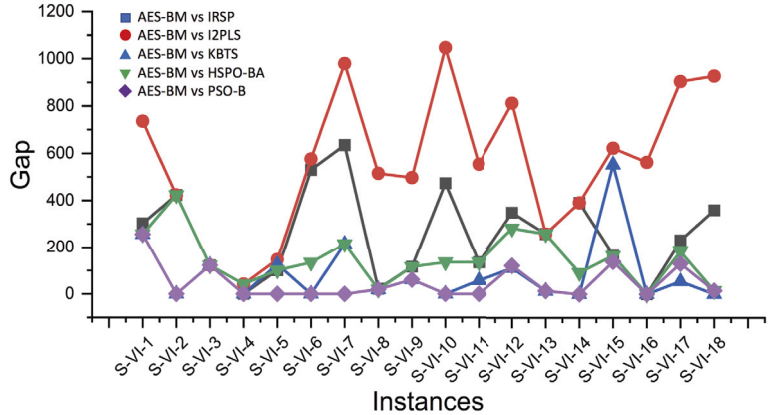
As demonstrated in [4,5], to emphasize the experimental study, particularly the assessment of which method performs better than another, a statistical analysis was employed. This analysis included both the *Sign test* and the *Wilcoxon signed-rank test*. In this study, we followed the same statistical analysis protocol to compare the results of AES-BM with those reached by other tested algorithms (I2PLS, KBTS, IRSP, HSPO-BA, PSO-B). We note that the Sign test evaluates the number of negative (< 0) and non-negative (> 0)



(a) Set_IV



(b) Set_V



(c) Set_VI

Fig. 6. Variations of the Gap achieved by AES-BM versus the five best methods in the literature: Group 2.

gaps in reached bounds when comparing two algorithms. On the other hand, the Wilcoxon signed-rank test provides an alternative quantitative analysis. In the comparison between two algorithms [4,5], denoted as A_1 and A_2, let LB_{A_1} (respectively, LB_{A_2}) represent the final bound of A_1 (respectively, A_2). The hypotheses are defined as follows: (i) H_0: $LB_{A_1} - LB_{A_2} = \alpha$, indicating that algorithm A_1 performs better than A_2 (for a maximization problem), and (ii) $\overline{H}_0$: Algorithm A_2 performs better than A_1, expressing the rejection of the first hypothesis H_0. Since the problem under consideration is a maximization problem, a greater bound and a higher number of better bounds indicate a better-performing algorithm.

Line 1 (and Line 2) of Table 7 reports the p-value associated with the Sign test (and the Wilcoxon signed-rank test). Line 3 reports the instances where the first algorithm dominates AES-BM, while line 4 indicates the instances where the first algorithm is dominated by AES-BM. Line 5 records the instances where both algorithms match the same bound. The table focuses particularly

Table 7
Behavior of AES-BM versus other compared methods: p-values for both the Sign test and the Wilcoxon rank test on the instances of the second group with a significance level of $\alpha = 0.05$.

	ρ_1	ρ_2	ρ_3	ρ_4	ρ_5
Sign test	1	< 0.0001	0.004	0.999	1
Wilcoxon Rank Test	1	< 0.0001	0.007	0.999	1
N^-	51	3	14	17	29
N^+	0	42	24	4	2
$N^=$	3	9	16	33	23

on the best provided bounds, where each column reports the comparative study between two consecutive algorithms A_1 and A_2. In this case, whenever A_1 dominates A_2, A_1 is compared to the next algorithm, namely A_3; otherwise, A_2 is compared to A_3. This process ensures that the resulting dominating algorithm is always compared to the next one until considering all tested algorithms. Sequential commentary on the results of Table 7 and the significance related to each used ρ follows (using a comparative study between the proposed method and the last five methods of the literature).

- Column 2 (ρ_1 = I2PLS vs KBTS): The p-value associated with ρ_1 is equal to 1, surpassing the significance level $\alpha = 0.05$ for both the Sign and Wilcoxon tests. This indicates that KBTS dominates I2PLS, supporting the rejection of the hypothesis H_0 for I2PLS in comparison to KBTS.
- Columns 3 (ρ_2 = KBTS vs IRSP): The p-value associated with ρ_2 is < 0.0001, smaller than the significance level $\alpha = 0.05$ for both the Sign and Wilcoxon tests (Lines 1 and 2). This indicates that KBTS significantly outperforms IRSP, supporting the acceptance of the hypothesis H_0 for KBTS in comparison to IRSP. In this case, one can observe that KBTS provides 42 best solution values (Line 4) out of the 54 instances when compared to those achieved by IRSP, representing a percentage of 77.78%.
- Column 4 (ρ_3 = KBTS vs HSPO-BA): The p-value associated with ρ_3 is 0.004 (resp. 0.007), which is smaller than the significance level $\alpha = 0.05$ for both the Sign test, line 1 (resp. Wilcoxon rank test), line 2. These indicate that KBTS outperforms HSPO-BA, supporting the acceptance of the hypothesis H_0 for KBTS in comparison to HSPO-BA. In this case, one can observe that KBTS provides 24 best solution values (Line 4) out of the 54 instances when compared to those achieved by IRSP, matches 14 best solutions and fails in 16 occasions.
- Column 5 (ρ_4 = KBTS vs PSO-B): The p-value associated with ρ_4 is 0.999, surpassing the significance level $\alpha = 0.05$ for both the Sign test (Line 1) and the Wilcoxon rank test (Line 2). This result indicates that KBTS is dominated by PSO-B, supporting the rejection of the hypothesis H_0 for KBTS in comparison to PSO-B. In this case, KBTS provides only 4 best solution values (Line 4) out of the 54 instances, in contrast to PSO-B, which achieves superior results in 17 instances and outperforms KBTS, representing a percentage of 31.48%.
- Column 6 (ρ_5 = PSO-B vs AES-BM): The p-value associated with ρ_5 is 1, exceeding the significance level $\alpha = 0.05$ for both the Sign test (Line 1) and the Wilcoxon rank test (Line 2). This indicates that PSO-B is dominated by AES-BM, robustly supporting the rejection of the hypothesis H_0 for PSO-B in comparison to AES-BM. In this case, the observation reveals that PSO-B only achieves 2 best solution values (Line 3) out of the 54 instances when compared to those attained by AES-BM, while it matches 23 best solutions and falls in 29 occasions.

Finally, Tables 6 and 7 collectively highlight AES-BM's superiority over all methods across the 54 instances. Moreover, AES-BM successfully identifies 27 new bounds (Column 8, Table 6, in bold space), constituting a substantial proportion of 50% in terms of new solutions.

5. Conclusion

In this paper, an adaptive scatter search-based method was introduced for tackling the set-union knapsack problem. The method initialized two sets: a reference set including elite solutions and a complementary set including diversified solutions obtained through a tailored greedy procedure. A cooperative approach was then employed, integrating alternative features. The initial stage focused on enhancing the reference set using adding and moving operators. To highlight the quality of elite solutions, a multi-tabu search-based strategy was incorporated. In the subsequent stage, the aim was to maximize the minimum Hamming distance between solutions to establish a new subset of diversified solutions, ensuring diversity and avoiding premature convergence. Finally, a fusion operator was employed to generate a series of solution subsets by merging various solutions from the reference set and the global set. These stages were integrated within an iterative search framework to improve solution quality. To evaluate the performance of the method, a series of experiments were conducted on instances extracted from the literature, comparing its provided results with those achieved by state-of-the-art methods. Promising results were obtained, with the method discovering more than 50% of new solutions, especially for large-scale instances, thereby confirming the effectiveness of the method in solving the set-union knapsack problem.

Finally, as the problem under investigation falls within knapsacking domain (Cutting, Packing and Knapsacking), there are various possibilities for further exploration of efficient algorithms. The first research direction involves the cooperation between mathematical programming and the method proposed in this work. In this case, we consider introducing some tailored valid constraints to isolate specific search spaces and thereby attempt to expedite the search for better solutions. A second research direction involves

the collaboration with machine learning, where the proposed algorithm can be considered as an improvement procedure within a learning process.

CRediT authorship contribution statement

Juntao Zhao: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Mhand Hifi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

The authors thank the anonymous referees for their helpful comments and suggestions which contributed to the improvement of the contents of this paper. The first author acknowledges the support from the China Scholarship Council (CSC No. 202306080131) and the University of Picardie Jules Verne.

References

[1] M. Aïder, O. Gacem, M. Hifi, Effect of learning strategies in an evolutionary method: the case of the bi-objective quadratic multiple knapsack problem, Neural Comput. Appl. 35 (2023) 1183–1209.
[2] I.M. Ali, D. Essam, K. Kasmarik, Novel binary differential evolution algorithm for knapsack problems, Inf. Sci. 542 (2021) 177–194.
[3] A. Arulselvan, A note on the set union knapsack problem, Discrete Appl. Math. 169 (2014) 214–218.
[4] I. Dahmani, M. Ferroum, M. Hifi, An iterative rounding strategy-based algorithm for the set-union knapsack problem, Soft Comput. 25 (21) (2021) 13617–13639.
[5] I. Dahmani, M. Ferroum, M. Hifi, Effect of backtracking strategy in population-based approach: the case of the set-union knapsack problem, Cybern. Syst. 53 (1) (2022) 168–185.
[6] I. Dahmani, M. Ferroum, M. Hifi, S. Sadeghsa, A hybrid swarm optimization-based algorithm for the set-union knapsack problem, in: 2020 7th International Conference on Control, Decision and Information Technologies, CODIT, vol. 1, IEEE, 2020, pp. 1162–1167.
[7] F. Glover, A template for scatter search and path relinking, in: European Conference on Artificial Evolution, Springer, 1997, pp. 1–51.
[8] F. Glover, J.-K. Hao, Efficient evaluations for solving large 0-1 unconstrained quadratic optimisation problems, Int. J. Metaheuristics 1 (1) (2010) 3–10.
[9] F. Glover, G.A. Kochenberger, B. Alidaee, Adaptive memory tabu search for binary quadratic programs, Manag. Sci. 44 (3) (1998) 336–345.
[10] F. Glover, M. Laguna, Tabu search, Springer, 1998.
[11] İ. Gölcük, F.B. Ozsoydan, Evolutionary and adaptive inheritance enhanced grey wolf optimization algorithm for binary domains, Knowl.-Based Syst. 194 (2020) 105586.
[12] O. Goldschmidt, D. Nehme, G. Yu, Note: on the set-union knapsack problem, Nav. Res. Logist. 41 (6) (1994) 833–842.
[13] Y. He, H. Xie, T.-L. Wong, X. Wang, A novel binary artificial bee colony algorithm for the set-union knapsack problem, Future Gener. Comput. Syst. 78 (2018) 77–86.
[14] M. Hifi, L. Wu, Lagrangian heuristic-based neighbourhood search for the multiple-choice multi-dimensional knapsack problem, Eng. Optim. 47 (12) (2015) 1619–1636.
[15] H. Kellerer, U. Pferschy, D. Pisinger, Knapsack Problems, Springer, 2004.
[16] M. Leconte, G.S. Paschos, P. Mertikopoulos, U.C. Kozat, A resource allocation framework for network slicing, in: IEEE INFOCOM 2018 - IEEE Conference on Computer Communications, INFOCOM, vol. 1, IEEE, 2018, pp. 2177–2185.
[17] G. Lin, J. Guan, Z. Li, H. Feng, A hybrid binary particle swarm optimization with tabu search for the set-union knapsack problem, Expert Syst. Appl. 135 (2019) 201–211.
[18] R. Marti, M. Laguna, F. Glover, Principles of scatter search, Eur. J. Oper. Res. 169 (2) (2006) 359–372.
[19] R. Merkle, M. Hellman, Hiding information and signatures in trapdoor knapsacks, IEEE Trans. Inf. Theory 24 (5) (1978) 525–530.
[20] F.B. Ozsoydan, A. Baykasoglu, A swarm intelligence-based algorithm for the set-union knapsack problem, Future Gener. Comput. Syst. 93 (2019) 560–569.
[21] F.B. Ozsoydan, İ. Gölcük, A reinforcement learning based computational intelligence approach for binary optimization problems: the case of the set-union knapsack problem, Eng. Appl. Artif. Intell. 118 (2023) 105688.
[22] G. Perboli, L. Gobbato, F. Perfetti, Packing problems in transportation and supply chain: new problems and trends, Proc., Soc. Behav. Sci. 111 (2014) 672–681.
[23] Z. Wei, J.-K. Hao, Iterated two-phase local search for the set-union knapsack problem, Future Gener. Comput. Syst. 101 (2019) 1005–1017.
[24] Z. Wei, J.-K. Hao, Kernel based tabu search for the set-union knapsack problem, Expert Syst. Appl. 165 (2021) 113802.
[25] Z. Wei, J.-K. Hao, Multistart solution-based tabu search for the set-union knapsack problem, Appl. Soft Comput. 105 (2021) 107260.
[26] C. Wu, Y. He, Solving the set-union knapsack problem by a novel hybrid Jaya algorithm, Soft Comput. 24 (2020) 1883–1902.